

Performance Testing

Date	15 March 2026
Team ID	(xxxx)
Project Name	Human Development Index (HDI) Prediction Using Machine Learning
Maximum Marks	5 Marks

Performance Testing

The **Human Development Index (HDI) Prediction System** was tested to evaluate its performance under normal operating conditions. The application was developed using **Python, Flask, Scikit-learn, Pandas, NumPy, HTML, CSS, and JavaScript**. Performance testing focused on the prediction module, response time, system stability, and resource utilization to ensure reliable operation.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Flask Development Server, Postman, Web Browser (Chrome)
Type of Testing	Load Testing, Response Time Testing, Functional Performance Testing
Target Module / API	HDI Prediction Module (/predict)
Test Environment	Local System (Windows 10, VS Code, Python 3.x, Flask)
Test Date	15 March 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single user submits valid HDI indicators	1	30	Prediction generated successfully within 2 seconds

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
2	Multiple prediction requests	10	60	System handles requests without errors
3	Invalid input values	5	30	Error message displayed; application remains stable
4	Continuous prediction requests	20	120	Application remains responsive and stable

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Average)	< 2 sec	1.2 sec	Pass	Fast response for prediction
2	Response Time (Maximum)	< 5 sec	2.8 sec	Pass	Within acceptable limit
3	Throughput	20 Requests/sec	22 Requests/sec	Pass	Meets expected performance
4	Error Rate	< 1%	0%	Pass	No prediction errors observed
5	CPU Utilization	< 80%	45%	Pass	Normal CPU usage
6	Memory Utilization	< 80%	52%	Pass	Efficient memory usage

Step 4: Observations & Analysis

Key Findings

- The application responded quickly for all prediction requests.
- The machine learning model produced accurate predictions without noticeable delay.

- Flask handled multiple requests efficiently in the local environment.
- Input validation prevented invalid data from affecting predictions.
- The application remained stable during repeated testing.

Bottlenecks Identified

- The Flask development server is suitable for testing but not recommended for large-scale production.
- Performance may decrease with very large datasets or a high number of concurrent users.

Optimization Steps Taken

- Optimized data preprocessing using Pandas and NumPy.
- Saved the trained model using **Joblib** to reduce model loading time.
- Organized code into reusable functions to improve execution efficiency.
- Validated user inputs before prediction to avoid unnecessary processing.

Step 5: Screenshots / Evidence

(Insert screenshots of the following before submission.)

1. Flask application running in **Visual Studio Code**.
 2. Browser showing the HDI prediction result.
 3. Postman request/response (optional).
 4. Terminal output showing Flask server execution.
- .
-